Batch: iitr-as-2603 · **Program:** Certification in Agentic Systems and Design — IITR-AS-2603 · **Institute:** IIT Roorkee

This guide compiles every concept from your course into one revision resource, organized in four parts: Python & Data Foundations, ML Algorithms & GenAI Basics, RAG & Agent Fundamentals, and Agent Ops & Deployment. Work section by section, use the tables for quick recall, and self-test with the **Key terms** line at the end of each section.

PART A — Python & Data Foundations

1. Python Basics, Environment, and Tooling

Python is beginner-friendly, English-like, and central to ML/AI work.

Environments: | Tool | Type | Notes | |—|—|—| | VS Code | Local editor | Explorer, Search, Source Control, Run & Debug, Extensions panes; needs Python + Jupyter extensions; supports `venv` for project isolation | | Google Colab | Cloud notebook | No install, free GPU/CPU, built-in AI assist |

- `.py` files = plain scripts; `.ipynb` = Jupyter notebooks (need kernel selection).
- AI coding aids (GitHub Copilot, Gemini, ChatGPT) speed up writing/debugging code — useful for learning, not a replacement for fundamentals.
- **API / API key**: an API is a contract letting software talk to other software (e.g., Uber → Google Maps). An API key is a secret token proving authorized access to a paid/secured API. Store keys in a `.env` file (never hardcode); add `.env` to `.gitignore`.
- **Git & GitHub** (version control):
  - Git tracks file changes over time; enables collaboration, history, rollback, independent feature branches.
  - Setup: `git config --global user.name "..."`, `git config --global user.email "..."`.
  - Core flow: **stage** changes → **commit** (snapshot + message) → **push** to GitHub.
  - **Branch**: independent copy of code for safe experimentation; can be published to GitHub.
  - **Pull**: download latest remote changes. **Pull Request (PR)**: request to merge one branch into another, used for review before integration.
  - GitHub repos can be **public** (anyone) or **private** (restricted).

Variables & operators: - Types: Integer, Float, String, Boolean, List (Tuple/Dictionary covered later). - Arithmetic: `+ - * / % ** //` · Comparison: `== != > < >= <=`.

Key terms: venv, API key, `.env`, git init/add/commit/push, branch, pull request, public vs private repo.

2. Control Flow: Conditionals, Loops

Decision making: - `if` executes a block only when condition is true; `if-else` gives two outcomes; `if-elif-else` chains multiple conditions (e.g., grading). - Indentation defines blocks in Python — missing/incorrect indentation is a syntax error. - `input()` returns a string by default; wrap with `int()`/`float()` to convert. - Logical operators: `and` (both true), `or` (either true), `not` (negate). Boolean expressions evaluate to `True`/`False`. - `len()` returns length of a string/sequence; `+` on strings concatenates.

Loops: | Loop | Driven by | Needs manual counter? | Best for | |—|—|—|—| | `while` | Condition | Yes — must increment or risk infinite loop | Unknown iteration count (e.g., input validation) | | `for` | Sequence (list/string/`range`) | No — auto-increments | Known/iterable sequence |

- **Loop invariant**: a property true before/after each iteration, used to reason about correctness/termination.
- `range(stop)` → 0..stop-1; `range(start, stop)`; `range(start, stop, step)` (step can be negative for descending).
- **`break`**: exits the nearest enclosing loop immediately (only the innermost loop in nested loops). **`continue`**: skips rest of current iteration, moves to next.
- **Nested loops**: loop inside a loop, for multi-dimensional data (matrices, grids, pixel traversal). Inner loop completes fully for each outer iteration.

```
count = 0
while count < 5:
    print("Hello Python"); count += 1
for char in "Python": print(char)
for i in range(3, 10, 2): print(i)     # 3,5,7,9
for i in range(1, 10):
    if i == 6: break
    print(i)
```

Key terms: if/elif/else, while vs for, range(), break, continue, nested loops, loop invariant.

3. Core Data Structures: Lists, Strings, Functions, Dictionaries, Classes

Lists — ordered, mutable, allow duplicates, can mix types (unlike arrays which are homogeneous). - Zero-indexed; negative indexing from the end (`-1` = last). - `append()` add, `remove()` delete by value, index assignment to modify. - **List comprehension**: `squares = [x**2 for x in numbers]`. - List functions: `append(item)`, `extend([items])`, `insert(index, item)`.

Strings — immutable sequences of characters. - `.lower()`, `.upper()`, `.replace(old,new)`, `.split()` (→ list of words). - Slicing `s[start:end]` (end exclusive); negative indices from end; `s[::-1]` reverses via step –1.

Functions — `def name(params): ... return value`. - `return` sends a value back to the caller (usable further); `print()` only displays to console, returns nothing usable. - Parameters are placeholders filled by arguments at call time.

Dictionaries — key-value pairs in `{}`; keys unique (duplicate keys overwrite). - Access: `d["key"]` (raises error if missing) vs `d.get("key")` (returns `None` if missing — safer). - Update/add: assign to a key. Loop: `for k, v in d.items():`. - Remove: `.pop(key)` (removes + returns value), `.popitem()` (removes last inserted pair), `del d[key]` (removes, no return). - Existence check: `'key' in d`.

Classes & Objects (OOP) - **Class** = blueprint (`class Computer:`); **Object** = instance (`com1 = Computer()`). - **Method** = function defined inside a class; **`self`** = first parameter of every method, refers to the instance itself. - **`__init__`** = constructor, runs automatically on object creation (no explicit call needed). - `type(x)` returns the class of `x` (e.g., `type(10)` → `int`); built-in types (`int`, `float`, `str`) are themselves predefined classes.

```
class Computer:
    def __init__(self):
        print("Computer is cool")
    def configure(self):
        print("Processor: i7, RAM: 16GB")
com1 = Computer(); com1.configure()
```

Key terms: list comprehension, negative indexing, `.get()` vs `[]`, `self`, `__init__`, method, `type()`.

4. NumPy — Fast Numerical Computation

- NumPy arrays are homogeneous (single dtype) → far faster/more memory-efficient than Python lists (10–100×); runs optimized C code, no Python loops.
- Adding two **lists** concatenates them; adding two **NumPy arrays** does element-wise addition.
- Install: `pip install numpy`; import: `import numpy as np`.
- Creation: `np.array([...])`, `np.zeros(size)`, `np.ones(size)`, `np.arange(start, stop, step)` (like `range` but returns an array; negative step for descending).
- **Shape**: tuple of dimensions — 1D `(n,)`, 2D `(rows, cols)`, 3D `(depth, rows, cols)`.
- **Reshape**: `.reshape(rows, cols)` — changes dimensions, total element count must stay constant.
- **Slicing**: `arr[start:stop]`; for 2D, `arr[row_start:row_stop, col_start:col_stop]`.
- Element-wise ops: `arr * 1.1` (scale), `arr ** 2` (square). NumPy is CPU-based (no GPU). Mixing int+float in an operation casts ints to float. `np.random.seed(42)` sets reproducibility.

Key terms: homogeneous dtype, element-wise ops, shape, reshape, slicing, `np.arange`.

5. Pandas — Data Manipulation

Basics - `import pandas as pd`; `df = pd.read_csv('data.csv')` loads a CSV into a DataFrame (table of rows/cols). - Exploration: `df.head(n)`, `df.tail(n)`, `df.shape` → (rows, cols), `df.columns`, `df.info()` (dtypes, non-null counts). - Nulls: `df.dropna()` (drop rows), `df.dropna(axis=1)` (drop cols), `df.isna()` (bool mask), `df.fillna(value)`.

Indexing | Method | Basis | Range behavior | |—|—|—| | `.loc[]` | Label-based | Inclusive of end label | | `.iloc[]` | Position-based | Exclusive of end index (like Python slicing); supports step |

Filtering / sorting - `&` = AND, `\|` = OR, e.g. `df[(df['age']>25) & (df['city']=='Delhi')]`. - `df.sort_values(by='col', ascending=False)`; multi-column: `by=['city','salary'], ascending=[True, False]`.

Creation, insertion, deletion - `pd.DataFrame(dict)`; add column: `df['col'] = [...]`; `df.loc[idx] = [...]` updates existing idx or inserts new row. - `df.drop(index=..., inplace=True)` / `df.drop(columns=[...], inplace=True)`; `inplace=True` mutates original. - Conditional drop: `df.drop(df[df['sales']<150].index, inplace=True)`. - `.rename(columns={'old':'new'}, inplace=True)`. - `.reset_index()` — resets to sequential 0..n-1, typically after grouping/dropping.

Aggregation & GroupBy - Aggregate fns: `sum, mean, count, min, max, median, var, std`. - `df.groupby('col')` groups rows; can group by multiple columns. - `.agg({'sales':'sum','units':'mean'})` — multiple aggregations, optionally renamed via dict. - Single-column groupby aggregation returns a **Series** — wrap in `pd.DataFrame(...)` before renaming columns.

Merging / Joining | Join type | Result | |—|—| | Inner | Only matching keys in both | | Left | All left rows + matches from right (NaN if unmatched) | | Right | All right rows + matches from left (NaN if unmatched) | | Outer | All rows from both; NaN where unmatched |

- `pd.merge(left, right, on='key', how='inner')`; differing key names: `left_on=..., right_on=...`; overlapping columns: `suffixes=('_left','_right')`.
- `.join()` is index-based by default, less flexible than `.merge()`.
- **Method chaining** lets you pipe merge → groupby → agg → reset_index → rename → sort_values in one expression.

Key terms: `.loc` vs `.iloc`, `groupby`, `.agg()`, merge types, `inplace`, method chaining.

6. SQL — Querying Relational Databases

Why databases: flat files (CSV/Excel) have row/column limits, no concurrent access, and weak data integrity. RDBMS (MySQL, PostgreSQL, SQL Server, Oracle) store data in tables with a **primary key** (unique row identifier).

Query structure & execution order:

`SELECT columns FROM table WHERE conditions GROUP BY cols HAVING agg_cond ORDER BY cols LIMIT n;`

Actual execution order: `FROM` (incl. JOINs) → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT`.

Clauses - `WHERE` filters rows (before grouping) using `=, !=, >, <, AND, OR, IN, BETWEEN`. - `ORDER BY` sorts (`ASC`/`DESC`); `LIMIT` caps row count (≈ Pandas `.head()`). - `GROUP BY` groups rows for aggregation; `HAVING` filters **groups** post-aggregation (must use an aggregate, e.g. `HAVING SUM(amount) > 10000`) — using `WHERE` on aggregated data is a common error.

Aggregate functions: `COUNT(*)` (all rows) vs `COUNT(col)` (non-null only), `AVG`, `MIN`, `MAX`, `SUM`.

Joins - **INNER JOIN**: only matching rows in both tables. - **LEFT JOIN**: all left rows + matches from right (NULLs where unmatched). - Primary keys (unique id) and foreign keys (reference to another table's primary key) enable joins; use table aliases for clarity.

Pandas ↔ SQL mapping: `SELECT/WHERE/ORDER BY/LIMIT` ≈ Pandas filter/sort/head; `pd.merge()` ≈ SQL JOIN; `pandasql` (`pip install pandasql`) lets you run SQL syntax directly on DataFrames (slower than native Pandas).

Data types: `INT`, `VARCHAR(n)`.

Key terms: primary key, foreign key, HAVING vs WHERE, JOIN types, query execution order, pandasql.

7. Data Visualization & Exploratory Data Analysis (EDA)

- **EDA** = investigating data to find patterns, anomalies, and test assumptions via stats + charts.
- Core Pandas EDA calls: `.head()`, `.info()`, `.describe()` (mean/median/counts).
- Missing values: remove rows, or impute (e.g., replace with mean) — choice depends on domain context.
- Data types: **numerical** (sales, spend) vs **categorical** (category names) — different analysis techniques.

Libraries: Matplotlib = basic/static plotting; **Plotly** = interactive (zoom, hover, dashboards). Both support the same chart types.

Chart types | Chart | Variables | Use | |—|—|—| | Line | Time (X) vs numeric (Y) | Trends over time (sales, indices) | | Bar | Categorical (X) vs numeric (Y) | Compare categories (bivariate) | | Histogram | One numeric (binned) | Distribution shape, skew, outliers (univariate) | | Scatter | Two numerics | Correlation between variables (bivariate) |

- **Outliers**: points markedly different from the rest. **Skewness**: right-skew = high-value outliers pull the mean right; left-skew = low-value outliers pull it left.
- Workflow: understand domain → inspect (missing/types) → clean → visualize (line/bar/hist/scatter as appropriate) → interpret/annotate.

```
import matplotlib.pyplot as plt
plt.plot(df['month'], df['sales'], marker='o'); plt.show()
plt.bar(df['category'], df['sales']); plt.show()
plt.hist(df['sales'], bins=10, edgecolor='black'); plt.show()
plt.scatter(df['study_hours'], df['marks']); plt.show()

import plotly.express as px
fig = px.scatter(df, x='funding', y='valuation', color='country'); fig.show()
```

Key terms: univariate vs bivariate, skewness, outlier, Matplotlib vs Plotly.

PART B — ML Algorithms & GenAI Basics

8. Machine Learning Fundamentals

- **Rule-based AI**: explicit if-else logic (washing machines, ACs, fire alarms) — fails on unstructured data (images) where rules are impractical.
- **ML**: learns patterns/rules from data instead of hardcoding them.
  - Traditional programming: `Data + Rules → Output`. **ML: `Data + Output → Learn Rules`.**
- **Supervised learning**: labels (Y) are known during training. Terminology: **Features (X)** = inputs; **Labels/Outputs (Y)** = target.
  - **Classification** predicts categorical outputs (default/no-default, spam/not-spam).
  - **Regression** predicts continuous numeric outputs (price, sales).
- **Train/test split**: typically 80/20. Training data = "textbook"; testing = "mock exam" (unseen).
  - `random_state` controls reproducibility of the split.
- **Overfitting**: high train accuracy, low test accuracy (memorized noise, doesn't generalize). **Underfitting**: low accuracy on both (model too simple/undertrained). **Good fit**: balanced, low error on both.
- **Baseline model**: simple reference — majority-class prediction (classification) or mean value (regression) — used to judge if a real model adds value.
- **Unsupervised learning**: no output labels; discovers structure/groupings (see §12).

Key terms: features/labels, train/test split, overfitting/underfitting, baseline.

9. Data Cleaning & Preprocessing

- **Garbage in, garbage out** — clean data before modeling. Common issues: missing values (NaN), logically-invalid values (negative price → fix via `abs()` or convert to NaN), duplicates.

Missing value imputation | Data type | Typical fill | |—|—| | Numerical | Mean or median of column | | Categorical | Mode (most frequent) | | Advanced | Predict via ML using other features |

- Choice must respect domain knowledge, not defaults blindly.
- **Duplicates**: identify with `df.duplicated()`, remove with `df.drop_duplicates()` — duplicates bias models by over-weighting repeated points.

Encoding categorical variables | Type | Example | Method | |—|—|—| | Ordinal (has order) | small<medium<large | Label encoding — integers reflecting order | | Nominal (no order) | car brand, fuel type | One-hot encoding — binary dummy column per category |

Scaling - Needed because features on different scales (age vs salary) can let large-magnitude features dominate training. - **Min-Max scaling**: `X_scaled = (X - min) / (max - min)` — bounds to [0,1]. Scale only numerical columns (not encoded categoricals). `sklearn.preprocessing.MinMaxScaler` with `fit_transform()`.

Preventing data leakage — the single most-tested preprocessing concept: - Leakage = test-set information influencing training → falsely optimistic results (analogy: seeing the exam paper before the exam). - Rule: split into train/test **first**; `fit_transform()` (imputer/scaler) **only on train**; on test, call `.transform()` only (never refit). - Applies identically to `SimpleImputer` and any scaler.

Class imbalance - One class vastly outnumbers another (e.g., 99% "no fraud" vs 1% "fraud"). A dummy always-majority model scores ~99% accuracy yet detects zero minority cases — the **accuracy paradox**. - Use **Confusion Matrix**: TP, TN, FP (Type I error), FN (Type II error, often the costlier miss e.g. disease/fraud). - **Precision** = TP/(TP+FP) — correctness of positive predictions. **Recall** = TP/(TP+FN) — coverage of actual positives. - **SMOTE** (Synthetic Minority Oversampling Technique): generates synthetic minority-class samples to rebalance; improves recall/F1 for the minority class. - **Cross-validation** (brief): multiple train/test splits averaged for a more robust performance estimate (like several mock tests).

Key terms: leakage, fit vs transform, one-hot vs label encoding, SMOTE, precision/recall, Type I/II error.

10. Regression Models

Linear Regression - Equation: **`y = mx + c`** — `m` = slope (Δy per unit Δx), `c` = intercept (y at x=0). - Multiple regression: `y = m1x1 + m2x2 + ... + c`. - **Residual** = Actual – Predicted; residual plots reveal fit quality/outliers. - `LinearRegression` from `sklearn.linear_model`; `coef_` = slope(s), `intercept_` = intercept. - Pipelines bundle scaling + model into one clean, leakage-safe workflow.

Regularization | Method | Penalty | Effect on coefficients | Use | |—|—|—|—| | Ridge (L2) | Sum of squared coefficients | Shrinks toward zero, never exactly zero | Reduce

overfitting, keep all features | | Lasso (L1) | Sum of absolute coefficients | Can shrink some coefficients to exactly zero | Feature selection + overfitting reduction |

- **Alpha**: regularization strength. `alpha=0` = plain linear regression; higher alpha = stronger shrinkage, risk of underfitting if excessive.
- **Polynomial features** add complex interactions → more dimensions → higher overfitting risk unless regularized.

Regression evaluation metrics | Metric | Meaning | |—|—| | R^2 (`model.score()`) | Proportion of variance explained; higher = better | | MAE | Average absolute error; intuitive | | RMSE | Root of mean squared error; penalizes large errors more than MAE |

Key terms: slope/intercept, residual, Ridge vs Lasso, alpha, R^2/MAE/RMSE.

11. Classification Models

Logistic Regression - Uses the **sigmoid function** to map linear output → probability (0–1). - `model.predict()` → hard label; `model.predict_proba()` → probability per class. - **Decision threshold** defaults to 0.5 but is tunable (e.g., lower threshold in fraud detection to catch more positives). - Binary (two classes) vs multi-class (>2 classes) classification.

Decision Trees - Flowchart of nested if-else rules from **root node** → **internal nodes** → **leaf nodes** (final prediction); path taken = **branch**. - Highly explainable ("white box"). **Max depth** hyperparameter: too shallow → underfit; too deep → overfit.

Random Forests - Ensemble of many decision trees on different data subsets; combines via majority vote ("wisdom of the crowd") → lower bias/variance than a single tree. - **`n_estimators`**: number of trees — too few underfits; too many doesn't cause overfitting, just wastes compute with diminishing returns. - Provides **feature importance** — which inputs drive predictions most.

Classification evaluation | Metric | Formula / meaning | |—|—| | Precision | TP/(TP+FP) | | Recall | TP/(TP+FN) | | F1 | Harmonic mean of precision & recall | | Accuracy | Overall correctness — misleading on imbalanced data | | ROC-AUC | Class-separation ability, robust to imbalance |

Random forests generally outperform single trees/logistic regression on accuracy and F1 in the course's case studies (loan approval, disease/cancer prediction, credit default).

Key terms: sigmoid, predict vs predict_proba, root/leaf node, max depth, n_estimators, ROC-AUC.

12. Unsupervised Learning

Clustering — K-means - Goal: group similar points without labels (customer segmentation, recommendations, anomaly detection). - **Steps**: (1) choose K, (2) init centroids randomly, (3) assign each point to nearest centroid, (4) recompute centroids

as mean of assigned points, (5) repeat until stable. - **Centroid** = mean position of a cluster's points. - **Choosing K — Elbow Method**: plot inertia (within-cluster sum of squared distances, WCSS) vs K; pick the "elbow" where extra clusters stop meaningfully reducing inertia. Too few K → high inertia; K = n points → inertia = 0 (overfit).

Dimensionality Reduction — PCA - Reduces many features to a few components capturing most variance — eases visualization and can improve downstream model quality by removing noise/redundancy. - Workflow: scale data (e.g. `StandardScaler`) → apply PCA → use reduced data for visualization/clustering/classification. - PCA before K-means: reduces noise, improves clustering accuracy, cuts computation.

Key terms: centroid, elbow method, inertia/WCSS, PCA, dimensionality reduction.

13. Time Series

- Rows represent successive time intervals; **order is critical** — shuffling invalidates the data.
- **Trend**: long-term consistent movement. **Seasonality**: regular repeating short-term pattern (e.g., AC sales peak in summer).
- **Train/test split must be chronological** — use `train_test_split(..., shuffle=False)` (e.g., train Jan–Aug, test Sep–Oct) to avoid leakage.
- **Feature engineering**: **Lag features** (past values as new columns, e.g. sales at t–1, t–2); **Rolling window/mean** (average of last n points, smooths noise, reveals trend, e.g. 7-day or 30-day moving average).
- **Evaluation**: standard R^2/MAE/RMSE plus **MAPE** = `|Actual − Predicted| / Actual × 100%` — percentage error, comparable across scales. Higher R^2 ↔ lower MAPE.
- Gradient Boosting typically outperformed Linear Regression on lag/rolling-window features in the course's case studies.

Key terms: trend, seasonality, lag feature, rolling mean, MAPE, `shuffle=False`.

14. Model Selection, Persistence & Deployment

- Compare multiple models side-by-side (accuracy, precision, recall, **F1**, plus training time/compute/latency) — accuracy alone misleads on imbalanced data.
- **Hyperparameter tuning**: manual sweep, or automated **`GridSearchCV`** (exhaustive search — computationally expensive).
- **Model persistence**: save a trained model for reuse without retraining.
  - **`joblib`** (preferred for sklearn models — faster, handles large arrays) vs **`pickle`** (slower, general-purpose). `joblib.dump()` / `joblib.load()`.
  - **Security**: never load an untrusted `joblib`/`pickle` file — arbitrary code execution risk.
  - Persist the **entire pipeline** (preprocessing + model), not just the model, for consistent predictions.

- **Gradio**: lightweight framework for a quick interactive prediction UI (sliders → model → output) — useful for demoing a persisted model to non-technical users.
- **Occam's Razor**: prefer the simplest model that fits well; add complexity only when justified. **Interpretability** matters in sensitive domains (healthcare, finance) — sometimes a shallower, explainable tree beats a stronger black-box model.

Key terms: GridSearchCV, joblib vs pickle, Gradio, Occam's Razor, interpretability.

15. Generative AI Fundamentals

- **Classical ML** predicts (regression/classification) from structured features; does not create new content.
- **Generative AI / LLMs** are trained on massive multimodal data (text, code, images) and generate new content by predicting the **next token** probabilistically.
- **Rule-based systems** are deterministic (same input → same output); **LLMs** are probabilistic (same prompt → outputs can vary).
- **Tokens**: smallest text unit for an LLM (not always a whole word); ~1 token ≈ 0.75 English words (~4 characters). Tool: `tiktoken` to count tokens (affects cost/speed/limits).
- **Context window**: max tokens (input + output) an LLM can process per call — its "working memory". Exceeding it can silently truncate/drop information. Chunk large documents before feeding them in.
- **Hallucination**: confident but fabricated/incorrect output — LLMs don't "know" facts, they output probability-weighted text; they also have a **knowledge cutoff** (no awareness of events after training).
- **RAG** (preview — full detail in §20) combines LLMs with retrieval + rules to ground answers and reduce hallucination.

Key terms: token, context window, hallucination, knowledge cutoff, next-token prediction.

16. Prompt Engineering

- **Prompt** = the instruction set given to an LLM; determines its output. Iterative refinement of prompts = **prompt engineering**.
- **Prompt structure**: **System message** (hidden high-level instructions: task, constraints, tone, output format) → **User message** (actual query) → **Assistant response** (model output).
- Vague prompts → generic outputs; specific/detailed prompts → tailored, accurate outputs. LLMs may lack niche/proprietary domain knowledge (e.g., internal error codes) and need extra examples.

Prompting techniques | Technique | Description | Best for | |—|—|—| | Zero-shot | No examples given | General tasks (translation, summarization) | | Few-shot | 1+ examples provided | Domain-specific/proprietary patterns | | Chain-of-thought (CoT)

| Model reasons step-by-step explicitly | Complex multi-step logic/arithmetic; costs more tokens — don't overuse on simple tasks |

- **Prompt template — five elements**: Role, Task, Instructions, Constraints, Output Format.
- **Correct layering order**: System message → Few-shot examples → Chain-of-thought → Output template.
- Token/cost trade-off: CoT and long few-shot blocks increase tokens → increase API cost and latency.

Key terms: system/user/assistant, zero/few-shot, chain-of-thought, prompt layering order.

17. LLM API Parameters & Providers

- **Temperature**: controls randomness. ~0 = consistent/deterministic (good for classification/structured tasks); ~1+ = creative/diverse (good for marketing copy).
- **Max tokens**: caps the length of the generated response (controls cost/verbosity).
- **Input tokens** (prompt) vs **output tokens** (completion) — both count toward billing and latency.
- **Context window limits** vary widely: older models ~4K–8K tokens; modern models (Claude, Gemini) up to ~1M tokens. Exceeding it can silently truncate input.
- Providers: **Groq** (hosts open-source models, free tier), **OpenAI** (proprietary GPT family, paid), **Anthropic** (Claude family) — switching providers usually only changes client instantiation, not the rest of the code.
- **Open-source / local LLMs (Ollama)**: run models locally for privacy/offline use; needs adequate RAM (~1B params ≈ 1GB RAM). `ollama pull <model>`, `ollama run`. Ollama's API is OpenAI-compatible, so switching cloud↔local can be just an endpoint change. Trade-off: cloud = easy/fast but network latency + privacy concerns; local = private/offline but needs hardware and setup effort.
- **Benchmarks** for comparing LLMs: MMLU, HellaSwag, BBH, ARC, GSM8K (math), HumanEval (code), TruthfulQA, SQuAD, DROP.

Key terms: temperature, max_tokens, context window, benchmark suite, local vs cloud LLM.

18. APIs, REST, and JSON

- **API**: contract letting software components communicate (waiter analogy — you order, it returns food).
- **REST API**: uses HTTP methods — `GET` (read), `POST` (create/update). Request components: URL/endpoint, method, headers (e.g., auth/API key), params (query filters). Response = status code + body (usually JSON).

Common status codes | Code | Meaning | |—|—| | 200 | Success | | 401 | Unauthorized (bad/missing API key) | | 404 | Not Found (wrong endpoint) | | 429 | Rate limit exceeded | | 500 | Server error |

- **JSON**: universal, human-readable, text-based data format; structurally similar to Python dicts. `json.loads()` (JSON string → dict), `json.dumps()` (dict → JSON string).
- LLMs alone cannot fetch live/real-time data (weather, stock prices) — must be combined with REST API calls; typical flow: call API → parse JSON → extract relevant fields → build a focused prompt → send to LLM (avoids wasting tokens on raw JSON).

```
import requests, json
response = requests.get(url, params={"latitude": 28.67, "longitude": 77.22})
if response.status_code == 200:
    data = response.json()
```

Key terms: GET/POST, status codes, JSON serialize/deserialize, live-data augmentation.

PART C — RAG & Agent Fundamentals

19. Retrieval-Augmented Generation (RAG) — Introduction

Why RAG: LLMs have a knowledge cutoff and no access to private/proprietary data, so they can hallucinate on such questions.

Three-step RAG process 1. **Retrieve** — fetch relevant chunks from an external knowledge base. 2. **Augment** — add retrieved chunks into the LLM prompt as context. 3. **Generate** — LLM answers grounded in that context.

Analogy: **open-book exam** — the LLM ("exam taker") consults a librarian (retriever) instead of answering from memory.

Pipeline components | Stage | What happens | |—|—| | Chunking | Split large docs into smaller pieces | | Embedding | Convert chunks to numeric vectors (semantic meaning) | | Vector DB | Store chunk text + embeddings + metadata | | Retrieval | Convert query → embedding → similarity search → top-k chunks | | Generation | LLM answers using retrieved chunks + query |

- **Enterprise use**: vector DB and retrieval often run **on-premise** for privacy (HR policies, financial reports); only the small retrieved context may go to a cloud LLM API.
- **Grounding rule**: answers must come only from retrieved documents — reduces hallucination.

Key terms: retrieve-augment-generate, grounding, on-premise vector DB.

20. Embeddings & Vector Search

- **Embeddings**: vector representations of text capturing semantic meaning (roots: Google's 2013 Word2Vec). Analogy: "Paris – France = Rome – Italy" captures relational meaning.
- **Semantic similarity** (meaning-based) beats keyword search for retrieval quality; **cosine similarity** is the preferred distance metric.
- **ChromaDB** (vector database used throughout the course): persists embeddings on disk; operations — `upsert` (insert/update), `query` (similarity search), `get` (exact lookup by ID/metadata), collections (≈ SQL tables) hold IDs, text, embeddings, metadata.
- **Metadata** (source, page number, category) enables filtering before/alongside similarity search — critical for traceability/auditability of retrieved answers.

Key terms: cosine similarity, upsert, collection, metadata filtering.

21. Chunking & Document Preparation

- **Chunking**: splitting large documents (e.g., a 100-page PDF) into smaller retrievable pieces.
- **Fixed-size chunking**: common default ~512 characters per chunk, with an **overlap** (e.g., 16 characters) so context isn't lost at chunk boundaries.
- **Chunk size trade-off**: too small → meaningless/poor-context embeddings; too large (whole doc as one chunk) → defeats retrieval precision. ~512 chars is a practical balance.
- **Semantic chunking** (advanced): groups sentences by embedding similarity rather than a fixed character count — preserves coherence, can reduce chunk count needed for accurate retrieval.
- **Metadata** per chunk: source document ID, page number, chunk index, category — vital for traceability/debugging/compliance.
- Real demo: a 130-page Tesla financial report → 351 chunks via LangChain's recursive character text splitter; a 600+-page ABB manual → 1,034 chunks.
- Embedding models used in the course: **all-MiniLM-L6-v2**, **GTE-large** — selection criteria include benchmark performance, cost, context window, throughput (see MTEB leaderboard for comparisons).

Key terms: fixed-size vs semantic chunking, overlap, MTEB leaderboard.

22. Retrieval & Grounded Generation

- **Top-k retrieval**: query → embedding → fetch the k most similar chunks. **k trade-off**: low k = fast but may miss context; high k = more context but more latency/noise/cost.
- **Grounded generation**: system prompt instructs the LLM to answer **only** from provided context, and to say "I don't know" rather than fabricate when context is insufficient.

- **Delimiters** (e.g., `##context`, `##question`) clearly separate context from the actual question in the final prompt.
- Retrieval quality is the most common RAG failure point — poor chunk retrieval → poor/irrelevant answers regardless of a strong LLM.

Key terms: top-k, grounded generation, delimiters, retrieval failure.

23. Memory & Control Flow in Chat Applications

- **Single-turn** (one Q&A, no context) vs **multi-turn** (conversation history retained, supports follow-ups).
- **Short-term memory**: conversation history kept for the current session (Python list/dict/JSON of messages), passed on every API call.
- **Long-term memory**: persistent facts about a user across sessions — often stored as vector embeddings in a small vector DB of "memory chunks" (e.g., ChatGPT's memory/personalization feature).
- **Persistence**: in-memory history is lost when the process ends; save to JSON/pickle on disk to survive across sessions.
- **Loop termination**: max turn limit, stop words ("exit"/"quit"/"stop"), or task-completion logic — prevents runaway loops and uncontrolled API cost.
- **Token management**: monitor with `tiktoken`; as history grows, **summarize periodically** to control token/cost growth.
- **Error handling**: wrap API calls in try/except; return user-friendly messages instead of raw API errors.

Key terms: short- vs long-term memory, persistence, loop termination, history summarization.

24. AI Agents & the ReAct Paradigm

- **Agent = LLM ("the brain") + Tools ("hands/legs") + System Prompt.** Unlike a plain LLM, an agent can *act* — invoke external functions/APIs, not just answer.
- **ReAct = Reason + Act**: loop of **Thought → Action → Observation**, repeating until the task resolves. The system prompt guides this reasoning loop.
- **Tools** = external functions the agent can call (Python runtime, search engine, database/SQL, browser control). Each tool needs a clear description ("resume") so the LLM knows when to use it.
- LangChain demo tools: Python REPL tool, Google Serper (search) API wrapper. `CreateReactAgent` builds the policy; `AgentExecutor` runs it; `verbose=True` shows the agent's thoughts/actions for debugging.
- **Cost/risk**: agents consume more tokens than a plain LLM call (repeated reasoning + tool calls) and can execute dangerous actions if unchecked (e.g., file deletion) — require safeguards.

- Real agent examples: Gemini (integrates Gmail/Drive/Photos), ChatGPT with tool integrations.

Key terms: Agent = LLM+Tools+Prompt, ReAct loop, AgentExecutor, tool description.

25. Prompt Versioning, Experiment Tracking, and Rate-Limit Handling

- **Prompt versioning**: track/reproduce/compare/rollback prompt changes — essential for production auditability.
  - File-based: `v1_system.txt`, `v2_system.txt`, … plus JSON config files (model params). Doesn't scale well.
  - **Registry pattern**: centralized dict/JSON mapping app+version → prompt text + config — avoids hardcoded paths.
- **MLflow**: open-source MLOps platform (Databricks, released 2018) for experiment tracking, model registry, versioning, and (for GenAI) auto-tracking LLM calls, prompts, latency, tokens, cost.
  - Core concepts: **Experiment** (container) → **Run** (one trial) → **Parameters** (hyperparams) → **Metrics** (R^2, F1, etc.) → **Artifacts** (models/plots) → **Model Registry** (versioned, stage-tracked models: dev/staging/production).
  - Auto-logging: one line, e.g. `mlflow.sklearn.autolog()`.
- **API rate limits**: `429` = rate limit exceeded; also `401` unauthorized, `500` server error.
- **Retry with exponential backoff**: increasing wait between retries (1s, 2s, 4s, 8s…); add **jitter** (randomized delay) to avoid synchronized retry storms across clients.
- **Tenacity** (Python library): clean decorator-based implementation of retry + backoff + jitter, simpler than manual try/except + sleep.

Key terms: registry pattern, MLflow Experiment/Run, exponential backoff, jitter, Tenacity.

26. Structured Outputs & JSON Schema

- **Problem**: LLMs are text-in/text-out; downstream systems (dashboards, databases) need structured, predictable fields, not free-form prose. The same fact can be phrased many valid ways (e.g., "81,461 million" vs "81.46 billion") — unusable without standardization.
- **JSON Schema**: formal blueprint/contract defining expected output structure.
  - **Required fields** (must be present), **properties** (field data types), **enum** (allowed value set), **constraints** (length, numeric range, format rules).
- **Four-step pipeline**: (1) **Prompting** — instruct the LLM to emit structured JSON; (2) **Schema definition** — the JSON schema file; (3) **Parsing** — clean LLM output (strip markdown fences/backticks) into a Python dict; (4) **Validation** — check required fields, enums, types, business rules before the data reaches downstream systems.

- Provider support: Groq's `response_format={"type": "json_object"}`; set **`temperature=0`** for deterministic structured output.
- Applies universally: RAG systems, agents, classification, extraction tasks.

Key terms: JSON schema, required/enum/constraints, parse-then-validate, temperature=0 for structured output.

27. End-to-End RAG Pipeline & Evaluation

Full pipeline recap: ingest → chunk (fixed-size or semantic) → embed → store in vector DB → retrieve → generate.

Cloud model deployment: Azure (Foundry/ML Studio), AWS (SageMaker/Bedrock) — similar to Groq in code structure, differing mainly in client setup/endpoint.

RAG Triad — evaluation metrics | Metric | Question it answers | Diagnoses | |—|—|—| | Groundedness | Is the response supported by retrieved context? | Fabrication / hallucination | | Answer Relevance | Does the response address the actual query? | Off-topic answers | | Context Relevance | Is the retrieved context relevant to the query at all? | **Retrieval failure — the most common RAG failure point** |

- **LLM-as-judge**: a separate, typically **more powerful** judge model rates outputs on a defined scale (e.g., 1–5) using explicit criteria; judge sees query + context + generated answer.
- **Multimodal document parsing** (for images/tables in PDFs): **LlamaParse** (commercial, free trial, auto text+image+table extraction) vs **PyMuPDF** (free/open-source, more manual coding).

Key terms: RAG Triad (groundedness/relevance/context relevance), LLM-as-judge, LlamaParse vs PyMuPDF.

28. Safety & Guardrails

- **Guardrails**: safeguard layers filtering inputs and validating outputs against safety/accuracy standards. Risk level depends on the application (poetry generation ≈ low risk; tool-access agents ≈ high risk).
- **Prompt injection**: adversarial input crafted to override system instructions and bypass safety rules (may look like nonsense to humans but alters model behavior via token manipulation).

Input sanitization - Toxicity detection (abusive/hostile language). - Prompt injection detection. - Tools: **Detoxify** (toxicity scoring), **LLM Guard** (toxicity + injection + bias + regex/secrets + sentiment, input and output scanners).

Output sanitization - **PII screening/masking** (addresses, phone, email) — e.g., confirm "delivered to address on file" instead of revealing the actual address. - **Bias detection**, **toxicity screening**.

Case study (e-commerce chatbot) — failures without guardrails: PII disclosure, internal process leakage (refund criteria), account-flag disclosure, inconsistent answers to rephrased questions. **Fix layers**: (1) explicit system-message rules (never disclose X, redirect to support), (2) input validation (injection + toxicity checks, reject on failure), (3) output validation (PII/bias/toxicity checks, block on failure).

Key terms: prompt injection, PII masking, Detoxify, LLM Guard, input vs output sanitization.

PART D — Agent Ops & Deployment

29. Agentic RAG

- **Static RAG**: retrieval is always triggered, predetermined. **Agentic RAG**: a ReAct agent autonomously *decides* whether/when to call RAG (or another tool) based on the query.
- **E-commerce case study — 5 tools**: SQL Tool (NL→SQL, SELECT-only safeguard against DML), RAG Tool (product/policy Q&A with citations), Feedback Tool (auto sentiment→rating), Defer-to-Human Tool (escalation logging), Days-Since Tool (utility example).
- **Tool implementation pattern**: write a Python function with a clear docstring → wrap with `@tool` decorator → the **docstring becomes the tool's description** for the LLM's decision-making.
- Agent maintains conversation history/context across the session; safety constraint example: SQL tool restricted to `SELECT` only.

Key terms: Static vs Agentic RAG, `@tool` decorator, SELECT-only SQL safeguard.

30. Multimodal AI

- **Key distinction**: Speech-to-Text (STT, e.g., Whisper) and Text-to-Speech (TTS, e.g., gTTS, 11 Labs) are deep-learning models, **not** LLMs. LLMs only process **text**.
- **Core principle**: convert every modality to text because text is the lossless universal bridge; audio/images can be noisy/degraded.
- **Three-step pipeline**: Audio → **STT (Whisper)** → Transcript → **LLM summarization** → Summary → **TTS (gTTS/11 Labs)** → Audio output.
- **Vision/image-to-text** ("image captioning") uses vision transformers — same encoder-decoder architecture as text transformers, trained on image-text pairs.
- **Transformer architecture**: introduced in *"Attention is All You Need"* (2017); encoder-decoder; originally text-to-text, now extended to audio-to-text, text-to-speech, image-to-text, text-to-image — same fundamental principles across modalities.

- **Chatbots (ChatGPT, Gemini) are orchestrated multimodal agent systems**, not single models — an orchestration layer routes each request (text/image/audio) to the right specialized model.
- **Voice cloning** (11 Labs): needs ~30–60 min of training audio; raises ethical/impersonation concerns.

Multimodal RAG (advanced) - **Query Expansion**: LLM generates multiple alternative phrasings of a query → broader retrieval coverage (handles imperfect user phrasing); adds latency/token cost. - **Re-ranking**: a separate cross-encoder model re-scores/reorders already-retrieved chunks by relevance, ensuring the best chunks lead the context window. - **CLIP**: creates a shared vector space for text and images, enabling text-query search over image collections. - Combined workflow order: query → expand → retrieve text chunks (semantic search) + retrieve images (CLIP) → re-rank text chunks → pass top chunks + images to LLM. - Guardrail example (multimodal recipe app): compares vision-extracted vs speech-transcribed ingredients for semantic alignment before generating — mismatched inputs are refused. - Real-world uses: insurance claim photo assessment, AI interview platforms (video + speech + eye-tracking), nutrition advisors with visual answers.

Key terms: STT/TTS ≠ LLM, transformer (encoder-decoder), query expansion vs re-ranking, CLIP shared embedding space.

31. Model Context Protocol (MCP)

- **The N×M problem**: N applications each needing custom integration code for M different tools/APIs → duplicated effort, integrations break on any API change, hard to scale/maintain.
- **MCP** (Anthropic, November 2024): a standardized protocol for how AI models **discover, call, and get structured results from tools** — "USB-C for AI tools." One MCP server, many clients reuse it without custom code.
- **MCP does NOT replace APIs** — it adds a **discovery** mechanism + a **uniform tool contract** on top of existing APIs.
- **What MCP does *not* change**: tool design quality, versioning discipline, uptime/reliability, error handling, authentication/guardrails/audit logging — all still required.
- **Architecture**: MCP Server (vendor-maintained, centralizes tools) ↔ Clients/Hosts (apps/IDEs) — all use the identical connection format (`type: mcp`, server label, server URL, optional auth header) regardless of which server they connect to.
- **Custom servers**: **FastMCP** (Python) — `@mcp.tool` decorator turns a function into an MCP tool.
- Public MCP server examples: Hugging Face, GitHub, Stripe, Google, Cloudflare, Zapier, Alpha Vantage (`mcpservers.org` lists them). Enterprise/private example: Zomato's internal MCP servers.

- Tool errors are returned to the agent/LLM (not swallowed) so it can reason, retry, or adapt.

Key terms: N×M problem, discovery + uniform contract, FastMCP, `@mcp.tool`.

32. LangGraph — Multi-Agent Workflows

Core concepts | Concept | Definition | |—|—| | Node | A unit of work (Python function) — from a simple `strip()` call to invoking a full agent/RAG pipeline | | Edge | Connection defining execution flow between nodes | | State | Shared, mutable data structure ("running memory") passed through the whole workflow; every node can read/update it |

Edge / workflow types | Type | Behavior | Example | |—|—|—| | Sequential | Node 1 → 2 → 3, strict order | Understand → check availability → confirm | | Parallel | Multiple nodes run simultaneously from a common state, then combine | Generate ad + review + tagline in parallel | | Conditional | A routing node picks exactly one path based on a condition | Detect genre → route to matching story generator |

Build steps: (1) define State class, (2) write node functions, (3) create `StateGraph(State)`, (4) `add_node()`, (5) `add_edge()` / `add_conditional_edges()`, (6) `compile()`, (7) `invoke()`.

Checkpointing & resume - **Checkpointing** = saving workflow state at intervals (like a video-game save point), so a failure doesn't force a full restart — saves time, tokens, and API cost. - Three building blocks: **Thread ID** (groups checkpoints for one conversation/job), **Checkpointer** (storage engine), **Persisted State** (state written outside process memory, e.g., to disk/DB). - **MemorySaver**: in-memory, dev-only, lost on process end. **SqliteSaver**: persists to a local SQLite file, survives restarts (production-appropriate for small-scale use). - SQLite checkpoint DB has two tables: **Checkpoints** (full state snapshots per thread/step) and **Writes** (incremental state updates per node). - **Resume flow**: keep the thread ID → restart app → load last checkpoint by thread ID → continue from there (no repeated work). - Checkpointing solves process restarts / mid-run interruptions; it does **not** fix tool execution bugs, logic errors, or external service failures — those need actual code fixes.

Timeouts, retries, and HITL (human-in-the-loop) - **Timeout**: max allowed time before an operation is canceled (prevents indefinite blocking). - **Retry**: re-run a failed operation. **Exponential backoff**: increasing delay between retries (1s→2s→4s…) to avoid overwhelming a struggling service. - **Per-node timeout** (different nodes may legitimately need different durations) vs **Global timeout** (caps the entire workflow to meet a business SLA, e.g., "booking must complete in 20s"). - Implementation: `timeout_wrapper` around API calls raising `futures.TimeoutError`; LangGraph's built-in **`RetryPolicy`** supports bounded retries (e.g., max 3 attempts) with exponential backoff configured per node. - **HITL (Human-in-the-Loop) pattern** — real case study (healthcare SQL assistant, 7 nodes: Generate SQL → Classify Query → HITL Approval

→ Decision → Execute SQL → Present Results → Audit Logging): - **Read** queries (SELECT) execute automatically. - **Write** queries (INSERT/UPDATE/DELETE) pause at an **approval gate** for human sign-off. - **Unsafe** queries (DROP/TRUNCATE/ALTER) are auto-rejected. - **Interrupt & resume**: execution pauses at the approval node, state is checkpointed, and resumes once a human approves/rejects — governance metadata (reviewer, timestamp, decision) is logged.

Key terms: node/edge/state, sequential/parallel/conditional, Thread ID, Checkpointer, MemorySaver vs SqliteSaver, RetryPolicy, HITL interrupt/resume.

33. Observability, Tracing & Evaluation (LLMOps)

- **Observability**: monitoring/understanding production behavior via structured logging. **Tracing**: following one request-response cycle end-to-end via a **trace ID** (analogy: courier AWB number), capturing prompts, responses, latency, cost, timestamps at every step.
- **Structured logging** (MLflow-style, systematic, searchable, comparable, reproducible, audit-friendly) vs **unstructured logging** (scattered strings — impossible to search/compare/reproduce/audit).
- **MLflow for GenAI**: auto-tracks LLM API calls, latency, token counts, cost; supports prompt-version comparison and LLM-as-judge quality evaluation.
- **ML vs GenAI use of MLflow**: ML tracks many algorithms/hyperparameters to find the best model; GenAI tracks prompt/config/tool variations to find the best configuration — one model, many task variants.

Golden sets & evaluation - **Golden set**: curated question→expected-output pairs used to evaluate an LLM app before production. Authored by developers from domain knowledge — must cover typical, edge, and **prohibited** cases (e.g., "never allow table deletion"). - **Data split**: LLM apps use roughly a **10/90 split** — 10% as few-shot examples in the system prompt, 90% as gold examples for evaluation (unlike ML's 80/20 train/test, since LLMs are already pretrained).

Task type	Examples	Evaluation approach
Classification	sentiment, ticket routing, intent	Exact match → accuracy, precision, recall, F1, confusion matrix
Generation	summarization, chat replies	With ground truth: **BERT Score** (semantic embedding similarity, 0–1); without ground truth: **LLM-as-judge** rating (e.g., 1–5 scale)

- Case studies: binary sentiment classification reached ~96.9% accuracy; an 8-category imbalanced patent-classification task dropped to ~46.9% — illustrating how **class imbalance directly hurts accuracy**.
- Best practice: **label encoding** (0/1 instead of full category text) reduces token usage/cost.
- **Versioning**: version the whole "harness" (prompt + tools + retrieval config + settings) as one release bundle (v1.0, v1.1…); evaluate every release against the golden set to catch regressions.

Key terms: trace ID, golden set, 10/90 split, BERT Score, LLM-as-judge, harness versioning.

34. LLM Cost & Business-First Model Selection

- **Wrong question**: "Which model should I use?" (jumping straight to benchmarks/pricing). **Right question**: "What happens to my business if the output is wrong?" — risk assessment must precede model choice.

Key business dimensions for model selection: Failure cost, Reversibility (can errors be caught/corrected downstream?), Volume, Latency sensitivity, Criticality, Output consumer (machine/professional/vulnerable population), Auditability, Input quality, Task complexity, Error propagation.

Pattern from the course's case studies: high failure cost + zero reversibility + regulatory/vulnerable-population impact (e.g., loan summarization, HR resume screening, government benefits screening) → premium reasoning models (e.g., "Opus-class with extended thinking") + human review; low failure cost + high reversibility + high volume (e.g., mass property-listing generation) → cheapest viable model + batch processing for aggressive cost optimization.

Cost reduction strategies | Strategy | Saving | Trade-off | |—|—|—| | Batch API | ~50% off tokens | Up to 24h latency — only for non-real-time jobs | | Prompt caching | ~90% off repeated input tokens | Short reuse window (minutes) | | Output optimization | Output tokens often ~75% of total spend | Trimming response length has outsized cost impact vs. trimming input | | Buffer | N/A | Always add 10–15% for retries/failures |

- **Unit economics**: compute cost **per output** (per listing, per assessment) = `(total token cost × price) ÷ number of outputs` — stays meaningful regardless of scale.
- **Total Economic Impact = Cost of Model + Cost of Errors.** A pricier model that sharply cuts error rate can have a *lower* total cost of ownership than a cheaper, error-prone one.
- **Cost calculation example** (illustrative, converted to INR): system prompt ≈145 tokens, ~83 tokens/user message, ~124 tokens/output, at scale (25,000 outputs/day): daily cost before optimization ≈ INR 21.24; after batch API ≈ INR 10.62; after prompt caching ≈ INR 7.41; cost per output (after prompt caching) ≈ INR 0.0003;

annual (at the post-caching daily rate) ≈ INR 2,704.65 (input ≈INR 0.80–3/million tokens, output ≈1.25–6× input cost, depending on model).

- **Proof-of-concept methodology**: build a small test app, run 30–50 gold examples, read *actual* token counts from the API's usage field (never estimate), add 10–15% buffer, and **re-measure costs after every significant prompt change**.

Key terms: failure cost/reversibility, batch API, prompt caching, unit economics, total economic impact.

35. Production Deployment: FastAPI & Streamlit

Why move beyond Streamlit/Gradio - Streamlit/Gradio couple UI + backend in Python — great for POCs, not production (enterprise frontends use React/Angular/Vue in JavaScript/TypeScript). - Production separates: **Frontend** (JS/TS UI) ↔ **API layer** (FastAPI/Express) ↔ **Backend** (Python business logic/AI) ↔ **Database** (SQL/vector DB).

FastAPI - Exposes backend Python logic (RAG/agents/models) as accessible endpoints for any frontend (analogy: a power socket — access without exposing internal complexity). - Endpoints defined via decorators: `@app.get()`, `@app.post()`; communication is JSON over HTTP. - **Lifespan context manager**: startup tasks (load models, init vector DB) run before `yield`; shutdown/cleanup runs after `yield`. - Typical flow: user question → POST `/ask` → backend retrieves from vector DB → LLM generates → JSON response → frontend renders. - **Forward Deployed Engineer (FDE)**: role blending Python backend + JS/TS frontend + DB + customer-facing integration skills.

Streamlit - Converts a Python script into a browser UI with no HTML/CSS — ideal for rapid prototyping, not production scale. - Key components: `st.set_page_config()`, `st.title()`, `st.text_input()`, `st.button()`, **expanders** (collapsible sections — great for showing RAG sources/traces), **sidebar** (nav/filters), `st.session_state` (state across reruns). - **Deployment**: `streamlit run app.py` locally (port 8501); **Streamlit Community Cloud** offers free hosting via GitHub-connected public URL for POCs. - **Real case study (Insurance Agent)**: modular repo (app/ for agent logic, database/ for vector store, utils/, data/, tests/, config); an 8-node LangGraph workflow (Parse Claim → Validate → Generate Query → Retrieve Policy → Recommendation → Price Check → Final Decision → Invalid-Claim branch) with conditional routing on validation/price-check failures. - **Advanced RAG in production**: query-complexity classification routes simple queries to direct retrieval and complex queries to multi-chunk retrieval with context compression — improves latency/token efficiency. - **Software practices**: **Scaffolding** (minimal working skeleton to iterate on) vs **MVP** (bare-essentials production release); keep prompts in separate files, secrets in `.env`, and maintain a tests/ folder with ground-truth examples.

Key terms: lifespan/yield, `@app.get/post`, expander, session_state, scaffolding vs MVP.

36. Semantic Caching & Final Production Concerns

- **Semantic caching**: stores question→answer pairs as embeddings so near-identical future questions skip the full RAG pipeline.
  - **Cache hit**: similar question found in cache → instant answer, no pipeline call. **Cache miss**: no match → full pipeline runs, then the new pair is cached.
  - **TTL (Time To Live)**: cache entries expire after a set duration (commonly 1–2 hours) and refresh.
  - **Cache size management**: balance storage cost vs. retrieval speed with a size cap.
  - Demo numbers: max cache size 100 pairs, TTL 60 min, cache hit ≈0.05s vs cache miss ≈1.5s (~30× latency improvement).
- **Rate limiting**: `429` errors signal limits; **global capping** (whole-system limit) vs **per-user/per-session capping** (stop one user monopolizing capacity); combine with exponential-backoff retries.
- **Multi-layer caching architecture** (production systems often stack several): local in-memory cache, distributed cache (e.g., Redis), semantic vector cache, API response cache — each serves a different latency/cost trade-off.
- **Prompt caching** (provider-level, e.g., OpenAI/Anthropic): up to ~90% cost reduction on identical repeated prompt segments within a short TTL window (Anthropic: 5 min–1 hr).
- **Spec-driven development**: detailed PRDs (40+ specific points — architecture, functional requirements, tech stack, constraints) dramatically improve AI-coding-assistant (e.g., GitHub Copilot) output quality versus vague prompts like "build a RAG."
- Example production stack (final course demo): Streamlit frontend, LangChain backend, PyPDF for document loading, fixed-size recursive chunking, all-MiniLM-L6-v2 embeddings, ChromaDB, Groq/Llama for generation, in-memory cosine-similarity semantic cache.

Key terms: cache hit/miss, TTL, global vs per-user rate cap, multi-layer cache, spec-driven PRD.

Coverage note

This guide distills all 51 lecture summaries cached for this evaluation (lecture IDs 142593 through 168833), spanning Python fundamentals through advanced agentic-systems deployment. Every `in_syllabus_subtopics` entry from `_subtopics.json` is represented above; topics flagged there as `mentioned_only` (e.g., Toyota GPT name-drop, Kaggle competition mentions, MOE/quantization asides) are intentionally folded

into their parent section's context rather than given standalone treatment, consistent with their shallow/one-line depth in the source summaries.